YouTube Presentation — Script & Storyboard

Project: Auditing and Repairing a Published Nepali ASR Model — A Speaker-Diversity Case Study on XLS-R **Module:** ST7088CEM — Artificial Neural Networks **Student:** Bimochan Raj Kunwar — Coventry ID: 17108924 **Target length:** ~7–9 minutes

Note: the video itself has not been recorded yet — this document is the script/storyboard to record from. Once recorded and uploaded, the YouTube link should replace the placeholder in the report's "Project Links" section.

Scene 1 — Title / Hook (0:00–0:40)

Screen: Cover slide — title, your name/ID, Softwarica/Coventry logo.

Narration: "Hi, I'm Bimochan Raj Kunwar. This is my ST7088CEM Artificial Neural Networks project: auditing a published Nepali speech recognition model, and then repairing it. There's a public Nepali ASR model on Hugging Face that claims a 5.97% word error rate — remarkably good for a low-resource language. I wanted to find out: does that number actually hold up on real, diverse speech? Spoiler — it doesn't, by a lot. Let me show you."

Scene 2 — The Problem (0:40–1:45)

Screen: Slide with the model name `gagan3012/wav2vec2-xlsr-nepali` and the 5.97% figure, next to a note "measured on OpenSLR-43 — single speaker."

Narration: "This model was benchmarked on OpenSLR-43 — a dataset recorded by a single female speaker. That's the problem: a benchmark on one voice tells you almost nothing about how the model handles different speakers, accents, or recording conditions. This is a common issue in low-resource ASR research generally, not just this one model. So I framed two research questions: RQ1 — does the published claim survive contact with genuinely diverse speech? RQ2 — if not, can I repair the model by fine-tuning it, and is any improvement genuine or just a measurement artefact?"

Scene 3 — Architecture Walkthrough (1:45–3:15)

Screen: Figure 1 from the report — the XLS-R/wav2vec2 architecture diagram (CNN feature encoder → transformer encoder → CTC head, frozen vs. fine-tuned coloring). Point/highlight each block as you talk.

Narration: "Here's the model architecture — XLS-R, a wav2vec2 variant. Raw audio goes through a convolutional feature encoder — seven conv layers that turn the waveform into a sequence of latent frames. That feeds a 24-layer transformer encoder, which builds

contextualised representations using self-attention. Finally a linear layer plus a CTC head produces per-frame character probabilities, decoded into Devanagari text.

The important part for this project: I kept the CNN feature encoder **frozen** the whole time — in both the zero-shot audit and the fine-tuning. When I fine-tune, I only update the transformer encoder and the CTC head, using the CTC loss. I also keep the checkpoint's own vocabulary — I'm repairing this exact model, not training a new one or swapping in a different architecture like Whisper, which I actually considered and dropped early on, because changing architecture would confound the comparison."

Scene 4 — Task 1: The Audit (3:15–4:30)

Screen: Table from Section 4.1 — Self-reported 5.97%, measured in-domain 4.91%, out-of-domain 62.30%.

Narration: "Task 1: I reproduced the published number in-domain and got 4.91% WER — actually slightly better, so the claim technically holds on its own data. Then I ran the exact same checkpoint, zero-shot, on OpenSLR-54 — a completely different, multi-speaker Nepali corpus, 160 speakers. WER jumped to 62.30%. That's a twelve-fold degradation. So RQ1's answer is no — the published number does not describe real-world, multi-speaker performance."

Scene 5 — Task 2: The Repair (4:30–5:45)

Screen: Table from Section 4.2 — original vs. fine-tuned, 4.91 → 16.40 / 62.30 → 38.17.

Narration: "Task 2: I fine-tuned that same checkpoint on a 15-hour, 160-speaker, speaker-disjoint subset of OpenSLR-54 — meaning zero speaker overlap between train, validation, and test. Out-of-domain WER fell from 62.30% to 38.17% — a 39% relative improvement. I'm also honest about the cost: in-domain WER rose from 4.91% to 16.40%, because the model is now adapted toward diverse speech rather than that one original speaker. That's an expected specialisation trade-off, not a bug."

Scene 6 — The Ablation: Why the Split Matters (5:45–7:00)

Screen: Table from Section 4.3 — disjoint 38.17% vs. leaky 32.55%.

Narration: "Here's the part I think matters most methodologically. I trained a second model, identical setup, identical budget — but on a **leaky** split, where each speaker's utterances are scattered randomly across train, validation, and test instead of being kept separate. That leaky model scored 32.55% — better than the properly disjoint model's 38.17%. But that's not a real improvement — it's leakage. The leaky test set isn't truly unseen, because the model has already partially learned those speakers' voices during training. This confirms that if I — or anyone else — evaluates without controlling for speaker overlap, the reported number is inflated. It's the

same failure mode that produced the original misleading 5.97% benchmark, just demonstrated at a smaller scale on my own data."

Scene 7 — Efficiency: The Honest Negative Result (7:00–8:00)

Screen: Table from Section 4.5 — FP32 vs INT8, size 1203.6MB → 48.5MB, latency 204.7ms → 335.2ms.

Narration: "Last piece: I also benchmarked deployment efficiency. Dynamic INT8 quantization shrank the model by 96% — from about 1.2GB to 48.5MB. But it was actually 1.64 times **slower** on this Apple Silicon hardware, not faster, because the fast quantized kernels aren't available outside x86. I'm reporting that honestly rather than only showing the size win — a deployment optimisation that looks good on paper doesn't automatically transfer to every platform."

Scene 8 — Conclusion (8:00–8:45)

Screen: Summary slide with the two RQs and headline numbers, GitHub link visible on screen.

Narration: "So: RQ1 — the published 5.97% claim does not generalise to diverse speakers, degrading to 62.30% WER. RQ2 — yes, fine-tuning genuinely repairs a large share of that gap, down to 38.17%, and the speaker-leakage ablation confirms that improvement is real, not a measurement artefact. Everything here — code, MLflow experiment logs, data manifests, and the full report — is public on GitHub, link on screen and in the description. Thanks for watching."

Screen text overlay: `github.com/Rbimochan/NSTT-Lite`

Production notes

- Record screen capture of the MLflow UI (Appendix C screenshots) briefly during Scene 4–5 if time allows, to visually back up the "tracked with MLflow" claim.
- Keep each on-screen table visible for at least 8–10 seconds before cutting — these are the evidentiary core of the video.
- Optional B-roll: a few lines of terminal output from `run_efficiency_benchmark.py` for Scene 7.
- After recording and uploading, replace the placeholder in `report.md`'s "Project Links" section (and this file's header note) with the real YouTube URL.